

Week 4 Progress Report

Implement log ingestion and normalization components.

Overview

This week focused on implementing the ingestion and normalization layer of the mini-SIEM system. The primary objective was to process heterogeneous log sources and convert them into a unified event schema suitable for detection and analytics in later phases of the project.

Two datasets were used:

- A Linux `auth.log` (syslog-based authentication logs)
- A JSON-based honeypot dataset containing HTTP and connection activity

The goal was not correlation at this stage, but rather to demonstrate the ability to ingest multiple log formats and normalize them into a standardized event model. These datasets are not sufficient for meaningful correlation, which requires logs originating from the same host environment.

System Architecture

The project was structured modularly to reflect real-world SIEM architecture. The directory structure separates ingestion, normalization, and schema modeling:

- **ingest/** – Responsible for reading raw log sources
- **normalize/** – Contains parsing and classification logic for each log type
- **schema.py** – Defines the standardized `NormalizedEvent` data model
- **main.py** – Orchestrates the ingestion and normalization pipeline
- **data/raw/** – Stores original log files
- **data/normalized/** – Stores structured normalized output

This separation of concerns improves maintainability, scalability, and extensibility. New log sources can be added without modifying the core event model.

Unified Event Schema

A standardized `NormalizedEvent` model was defined to abstract away source-specific log formats. Each event includes structured fields such as:

- timestamp
- source
- event_type
- outcome
- src_ip
- user
- program
- tags
- raw_log
- extra

This unified schema ensures that detection logic in later phases can operate on structured semantic fields rather than raw log text.

Auth Log Normalization

The `authlog_parser.py` module processes unstructured syslog entries from Linux authentication logs.

Key implementation details:

- Regular expressions extract timestamp, host, program, PID, and message content.
- Specific patterns classify events into semantic categories such as:
 - `ssh_invalid_user`
 - `ssh_disconnect`
 - `ssh_no_identification`
 - `cron_session_open`
 - `ssh_socket_error`
- Each log entry is converted into a structured `NormalizedEvent`.

This transformation converts raw authentication strings into semantically meaningful security events.

Honeypot Log Normalization

The `honeypot.py` module processes structured JSON logs that contain nested payload objects.

Key implementation details:

- Parsing Mongo-style timestamp fields.
- Extracting attacker IP addresses and ports.
- Handling multiple event structures:
 - HTTP request events
 - Amun connection events
- Identifying suspicious command patterns (e.g., `wget`, `curl`, `bash`).
- Mapping events into the same unified schema used for auth logs.

This demonstrates normalization across heterogeneous formats. Both unstructured syslog and structured JSON are converted into a consistent event model.

Output and Validation

The ingestion and normalization pipeline successfully processed:

- 1,081,531 total events
- 0 parse failures

Normalized events were written to:

`data/normalized/events.jsonl`

Here are two example outputs in that file, one from `auth.log`, and one from `honeypot.log`:

```
{ "event_id": "de39dfa5-e3d0-4226-a078-c3b5399530f5", "timestamp": "2026-11-30T06:39:00+00:00", "source": "authlog", "event_type": "cron_session_close", "outcome": "success", "host": "ip-172-31-27-153", "user": "root", "program": "CRON", "pid": 21882, "message": "pam_unix(cron:session): session closed for user root", "tags": ["cron", "session"], "raw_log": "Nov 30 06:39:00 ip-172-31-27-153 CRON[21882]: pam_unix(cron:session): session closed for user root" }
```

```
{
  "event_id": "6313259a-66b5-43c5-8f6d-d2cb13a90779",
  "timestamp": "2015-05-15T16:04:54.747000+00:00",
  "source": "honeypot",
  "event_type": "amun_connection",
  "outcome": "unknown",
  "src_ip": "66.191.136.146",
  "src_port": 18535,
  "tags": ["honeypot", "connection"],
  "raw_log": "{ \"_id\" : { \"$oid\" : \"555619269f8c6d649a2a719f\" }, \"ident\" : \"eb030eb8-3c69-11e4-9ee4-0a0b6e7c3e9e\", \"timestamp\" : { \"$date\" : \"2015-05-15T16:04:54.747+0000\" }, \"normalized\" : true, \"payload\" : \"{\\\"attackerPort\\\": 18535, \\\"victimPort\\\": 110, \\\"victimIP\\\": \"172.31.13.124\", \\\"attackerIP\\\": \"66.191.136.146\", \\\"connectionType\\\": \\\"initial\\\"}\" }\", \"channel\" : \"amun.events\" }",
  "extra": {
    "channel": "amun.events",
    "ident": "eb030eb8-3c69-11e4-9ee4-0a0b6e7c3e9e",
    "victimIP": "172.31.13.124",
    "victimPort": 110,
    "payload": {}
  }
}
```

Event-type distribution output confirmed successful classification across both datasets. This validates the correctness of the normalization layer.

Below is a screenshot of the codes output:

```
=== Mini SIEM Week 4: Ingestion + Normalization ===
Normalized events written to: data/normalized/events.jsonl
Total events: 1081531
Parse failures: 0

Top event types:
  honeypot_event: 616176
  amun_connection: 371317
  ssh_disconnect: 46810
  authlog_other: 28867
  ssh_invalid_user: 12223
  http_request: 7199
  ssh_socket_error: 2403
  ssh_connection_closed: 1974
  ssh_no_identification: 969
  cron_session_close: 797
PS C:\Users\trent\OneDrive\Desktop\mini_siem>
```

Dataset Limitations and Rationale

The two datasets used this week were intentionally selected to demonstrate heterogeneous ingestion and normalization.

However, they are not suitable for final correlation and accountability analysis because:

- The auth.log dataset contains authentication activity.
- The honeypot dataset contains external HTTP and connection attempts.
- The two sources do not represent activity from the same system or host environment.
- There is no shared context (e.g., matching hostnames, shared user accounts, correlated timestamps).

Because of this, meaningful correlation between authentication events and system audit activity would not be possible using these datasets.

These sources were used solely to complete this week's objective of demonstrating multi-source ingestion and normalization. For the final phase of the project, datasets from the same system environment (e.g., auth logs and audit logs from a single host) will be required to support correlation and accountability analysis.

Summary

This week established the core ingestion and normalization foundation of the mini-SIEM system. The system now supports multi-source log ingestion, semantic classification, and structured event modeling — critical components of any modern SIEM architecture.

The project is now positioned to transition from data preparation into detection and correlation development.

Source for used datasets:

[SecRepo - Security Data Samples Repository](#)